Quelle: https://www.flickr.com/photos/nicola_s/20141007433

die Mobiliar

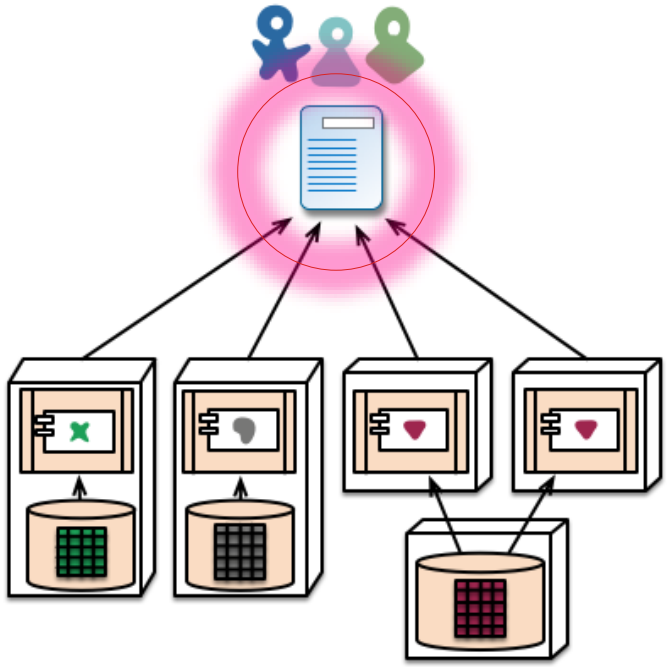
Commands & Events – Divide and Conquer in Microservice Architekturen

Beat Winistörfer, IT Architekt

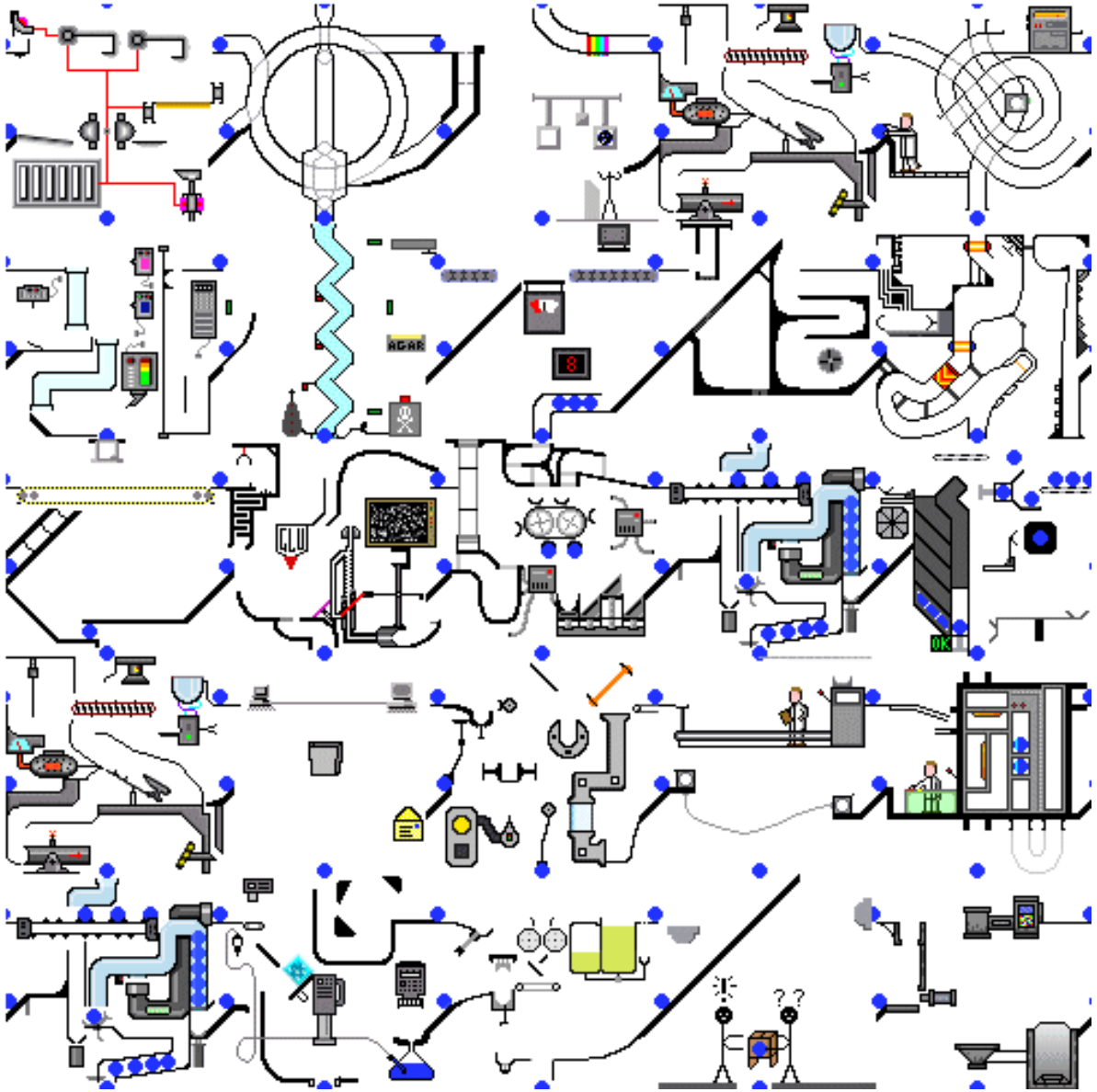
Aufteilen liegt im Wesen jeder Microservice Architektur

Microservice Architektur
Komplexe Anwendungssoftware als Suite von kleinen, unabhängigen Services mit eigener Datenhaltung.

Aber nicht alle Anforderungen machen an der Grenze der Microservices halt…



Quelle: https://martinfowler.com/articles/microservices.html



Quelle: http://insureblog.blogspot.com/2011/06/ode-to-rube-goldberg.html

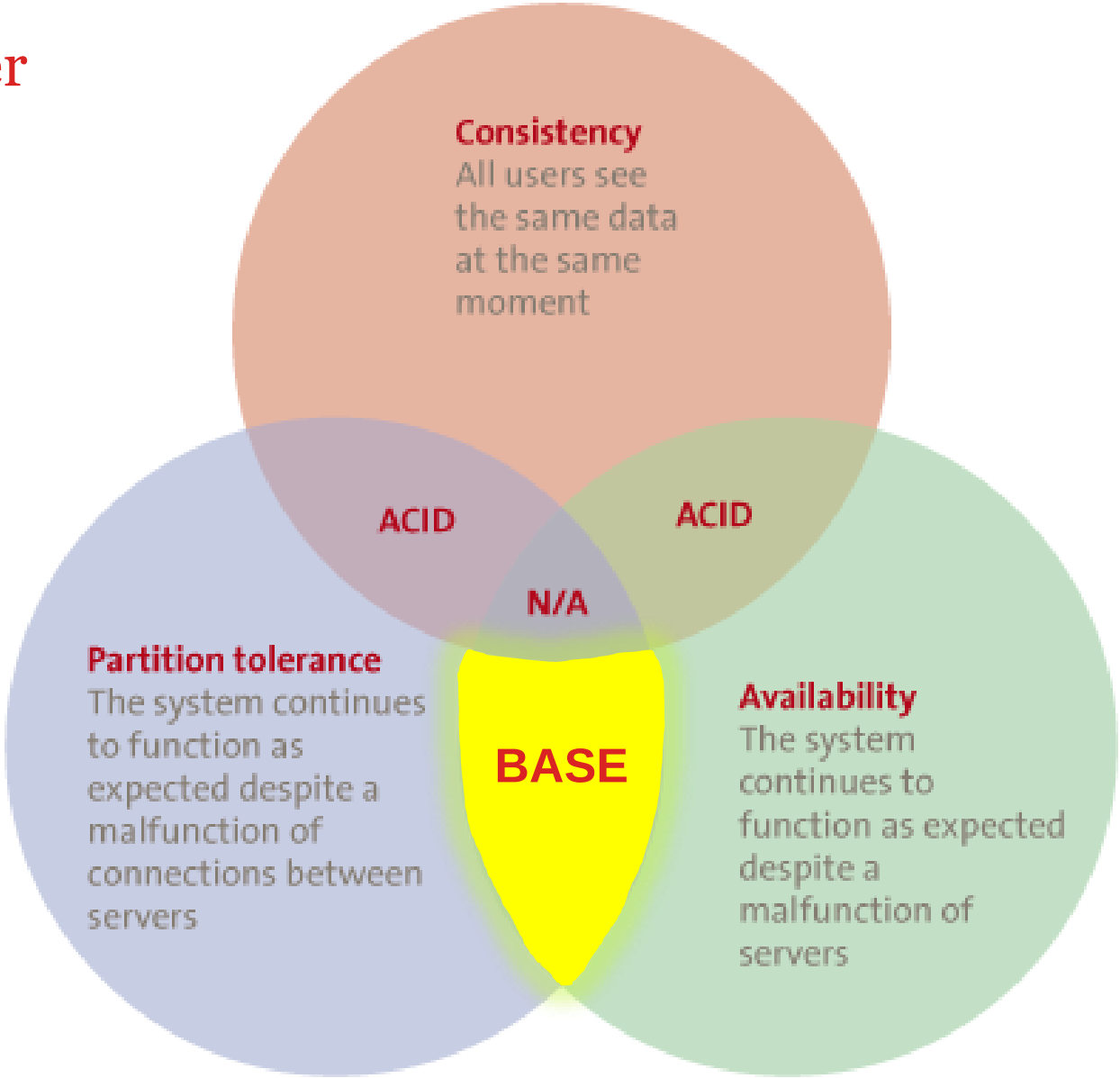
Das CAP Theorem von Eric Brewer

Ein verteiltes System kann nur 2 der folgenden 3 Eigenschaften erfüllen

1. Konsistenz
2. Verfügbarkeit
3. Ausfalltoleranz

BASE Prinzip

Basically **A**vailable
Soft State,
Eventual Consistency



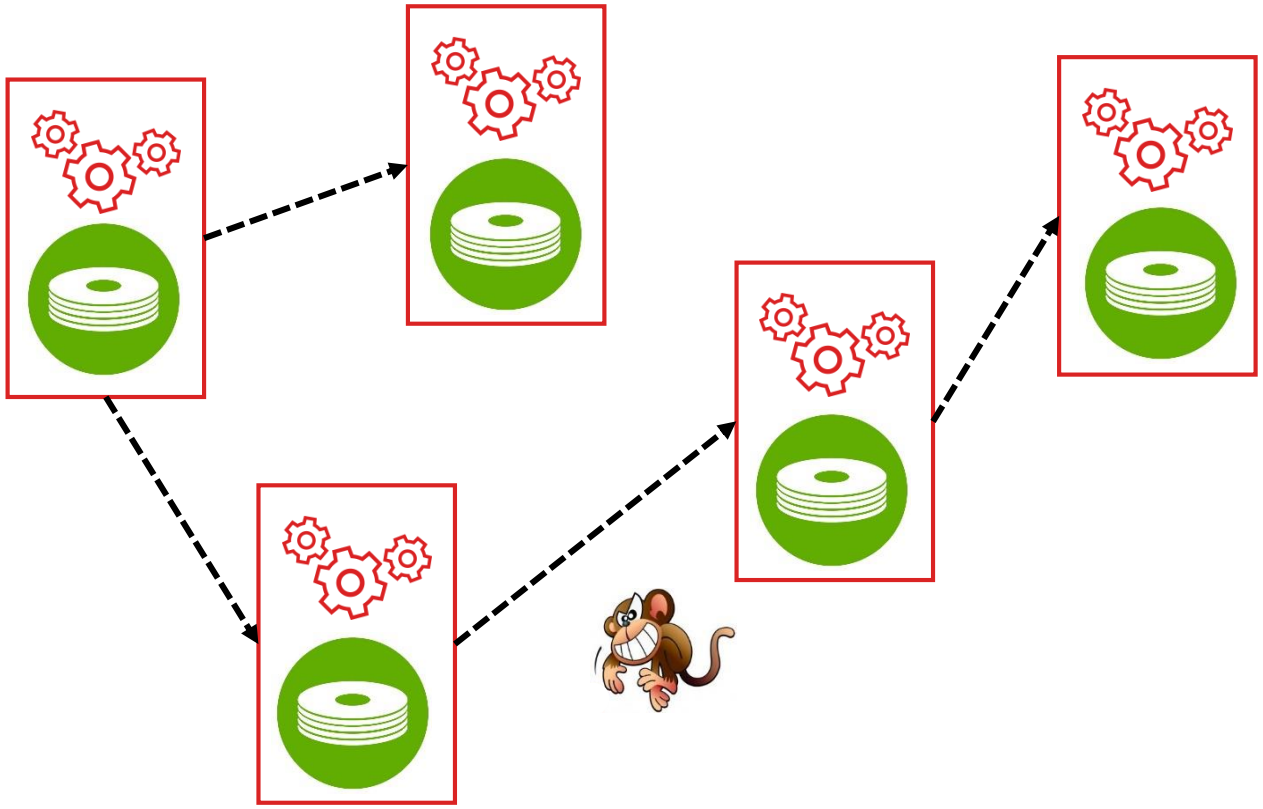
Quelle: https://www.compact.nl/en/articles/big-data-too-big-to-ignore-2/

BASE Transaktion als Abfolge von Statusänderungen in Microservices

Basically Available
Asynchrone Kommunikation überbrückt Verfügbarkeitsprobleme. Auftrag entgegen nehmen, Antwort später…

Softstate
Zustand in dem ein Teil der Microservices im neuen Zustand und ein Teil noch im alten Zustand sind.

Eventual Consistent
Schlussendlich erreichen alle involvierten Microservices einen konsistenten Zustand oder über Kompensationen wieder den Ausgangszustand.



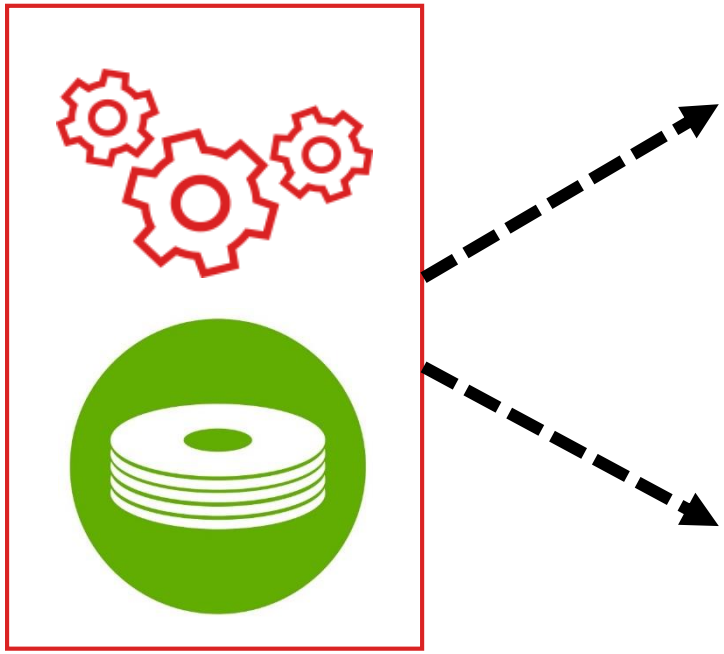
Den Unterschied zwischen "eventual consistent" und "eventuell konsistent" macht verlässliche Kommunikation

Statusänderung und sich daraus ergebende Kommunikation müssen eine Einheit bilden

- ACID Transaktion garantiert At-Least-Once Delivery beim Producer
  - Message-Tabellen & Hintergrund Versandjob
  - Kafka Transaktion
    (eigene Änderung & Kommunikation)

- Asynchrone Kommunikationsinfrastruktur muss ebenfalls At-Least-Once Delivery garantieren

- Consumer muss Idempotenz bei der Verarbeitung garantieren.



Commands & Events

- Kommunikationen aus Statusänderungen lassen sich grob in 2 Kategorien unterteilen

- **Events**: Benachrichtigungen aus der Vergangenheit über einen bestimmten Vorfall
  - Empfänger aus Sicht Produzent irrelevant. Benachrichtig
  - Mehrere Empfänger durchaus üblich, ändert sich über die
  - Kein Empfänger oft zulässig (Broadcast von Änderungen Problem sein (bspw. Dokument Eingang um den sich nie
  - Normalerweise nur ein Produzent

Beispiele:
- **Änderungs-Events**
- **Dokument-Eingang**
- **Prozess Abschluss**
- **Online Schadenmeldung**
- **…**

- **Commands**: in Zukunft zu erledigende Aufträge
  - Produzent hat eine klare Erwartung, muss aber die konkr Benachrichtigung über Erledigung gängig.
  - Normalerweise genau ein Empfänger aber durchaus meh
  - Kein Empfänger unzulässig (Fehler)

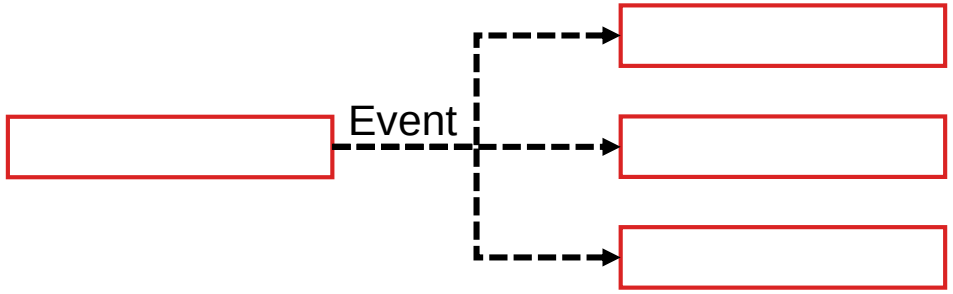
Beispiele:
- **Änderung durchführen**
- **Visum einholen**
- **Zahlung auslösen**
- **Dokument versenden**
- **…**

Prozessmodellierung und Verantwortlichkeit
Command oder Event?

Command – Auftraggeber bleibt in der Verantwortung
Der Auftraggeber bleibt in der Prozessverantwortung. Der Prozess ist erst abgeschlossen wenn der Auftrag erledigt wurde, allfällige Konsequenzen aus einer nicht möglichen Verarbeitung des Auftrags müssen vom Aufraggeber gezogen werden.

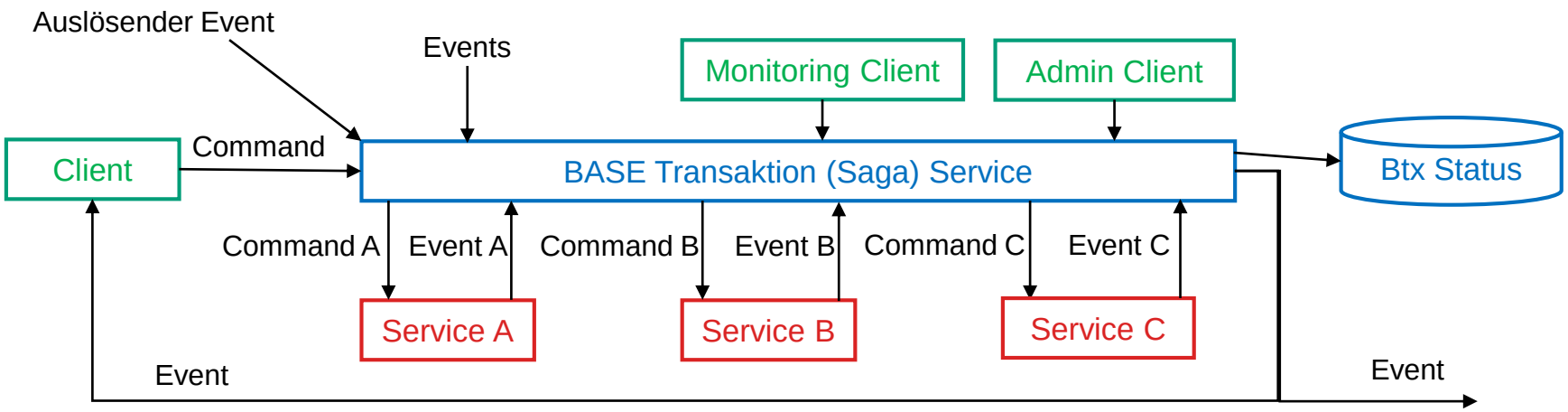
Orchestrierung

Event – Aprés moi le déluge… Verantwortlich für eine Verarbeitung ist der Konsument
Die Prozessverantwortung ist mit der Kommunikation der sich daraus ergebenden Events wahrgenommen.



Choreografie

Umsetzung von BASE Transaktionen als orchestrierender Microservice (Saga Pattern)



- BASE Transaktionen als Microservice
  - dezentral, CI/CD
  - Devops Verantwortlichkeit für Prozess
- Verschachtelbar (über Orchestrierung oder Choreografie), weitere Events integrierbar
- Deklarative Prozessdefinition
- Standard Schnittstellen für Überwachung und Administration

BTx @ Mobiliar

- **Java Library für Command/Event Erstellung/Verarbeitung**
  - Standard Datenbanktabelle für Commands/Events
  - Backgroundjob
  - Monitoring/Admin REST Interface

- **Java Library für**
  - Standard Daten
  - Fluent API für P
  - Backgroundjob
  - Prozessvisualisi
  - Monitoring/Admin REST Interface

- **Kommunikation**
  - Command Map
  - Events über JM
  - **Commands &**

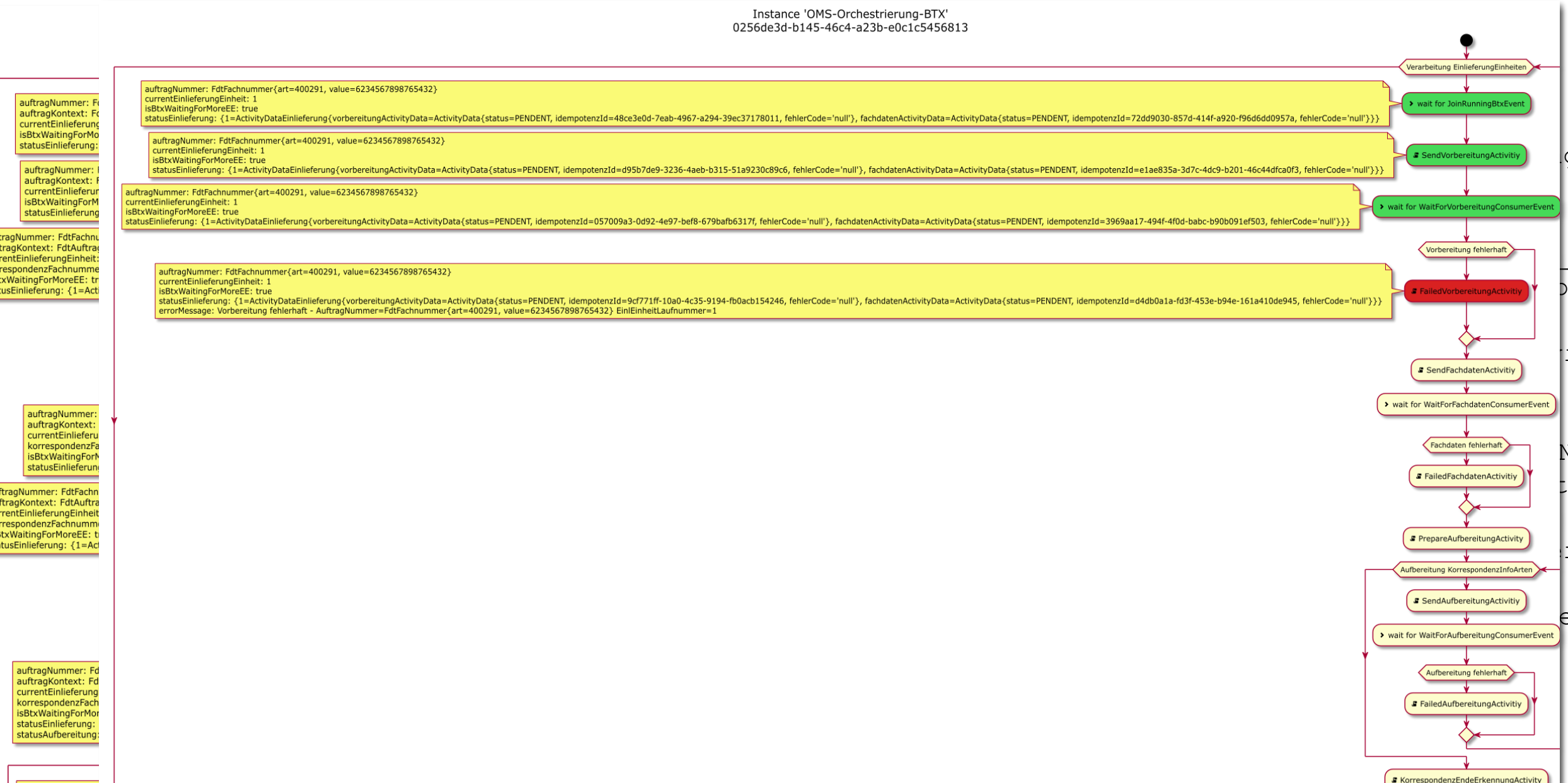
JES7 - Commandsbrowser Show/Hide | List Operations | Expand Operations

GET	/jap/commands	Lesen von fehlgeschlagene Commands
GET	/jap/commands/all	Lesen von alle Commands egal von status
PUT	/jap/commands/{commandId}	Fuehrt einen Command nochmals aus

JES7 - BTX-Admin Show/Hide | List Operations | Expand Operations

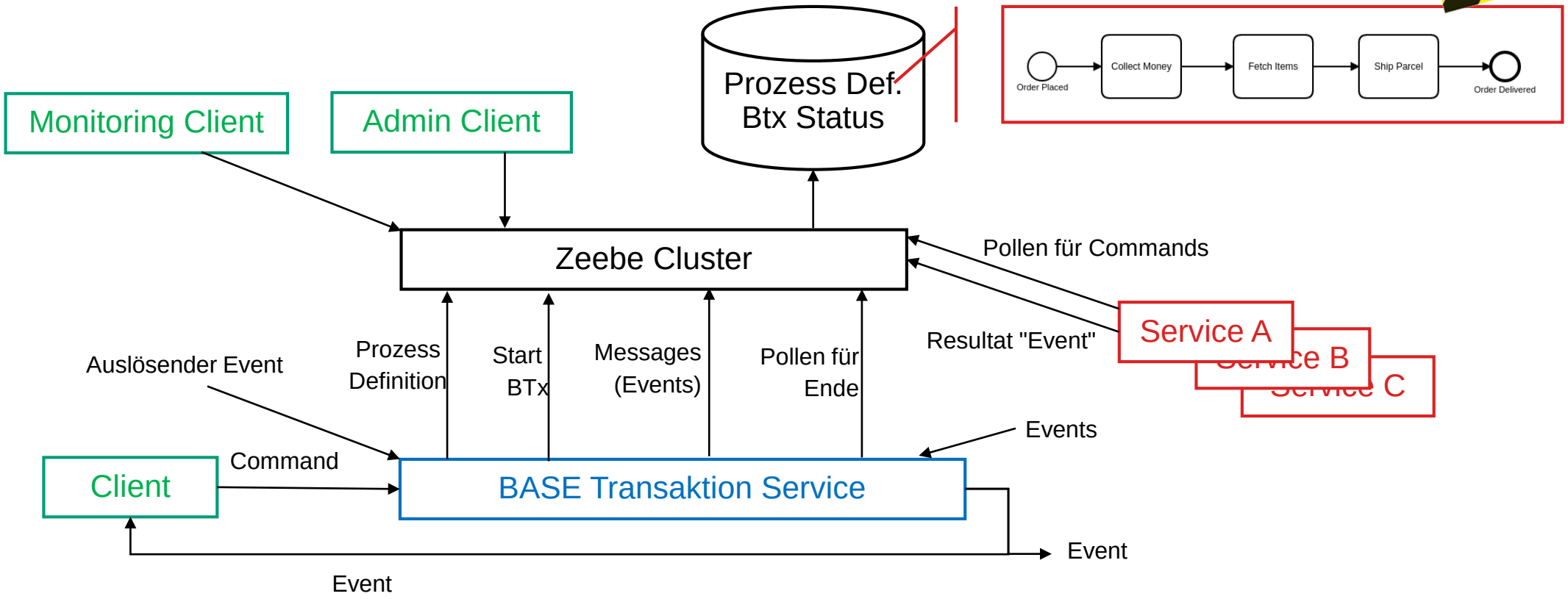
GET	/jap/btx/admin/definitions/all	all definitions
GET	/jap/btx/admin/definitions/{definitionId}	BTX-definition as SVG-XML
GET	/jap/btx/admin/instances/all	BtxInstances
GET	/jap/btx/admin/instances/incomplete	Incomplete instances
GET	/jap/btx/admin/instances/{btxId}	instance as SVG-XML

Processdefinition as Code, Visualisierung über PlantUML



Andere BTx Implementationen…

- Open Source
- Prozessdefinitionen in BPMN (reduziert)
- Commands Pollen und Resultat (gRPC bidirektional)
- Zusätzliche Messages können integriert werden



Andere BTx Implementationen…

- Cloud basiert
- Optimiert für Inter Cloud Kommunikation (HTTP Polling)
- Keine Prozess Definitionen
- Message Integration über Signale
- Progress Meldungen der Worker Services mit Abbruch Option

Admin Client

Monitoring Client

Btx Status

AWS SWF

Pollen für Commands

Progress

Resultat "Event"

Service A

Service B

Service C

Start BTx

Pollen Für Decision Tasks

Next Decision

Signal (Events)

Auslösender Event

Events

Command

Client

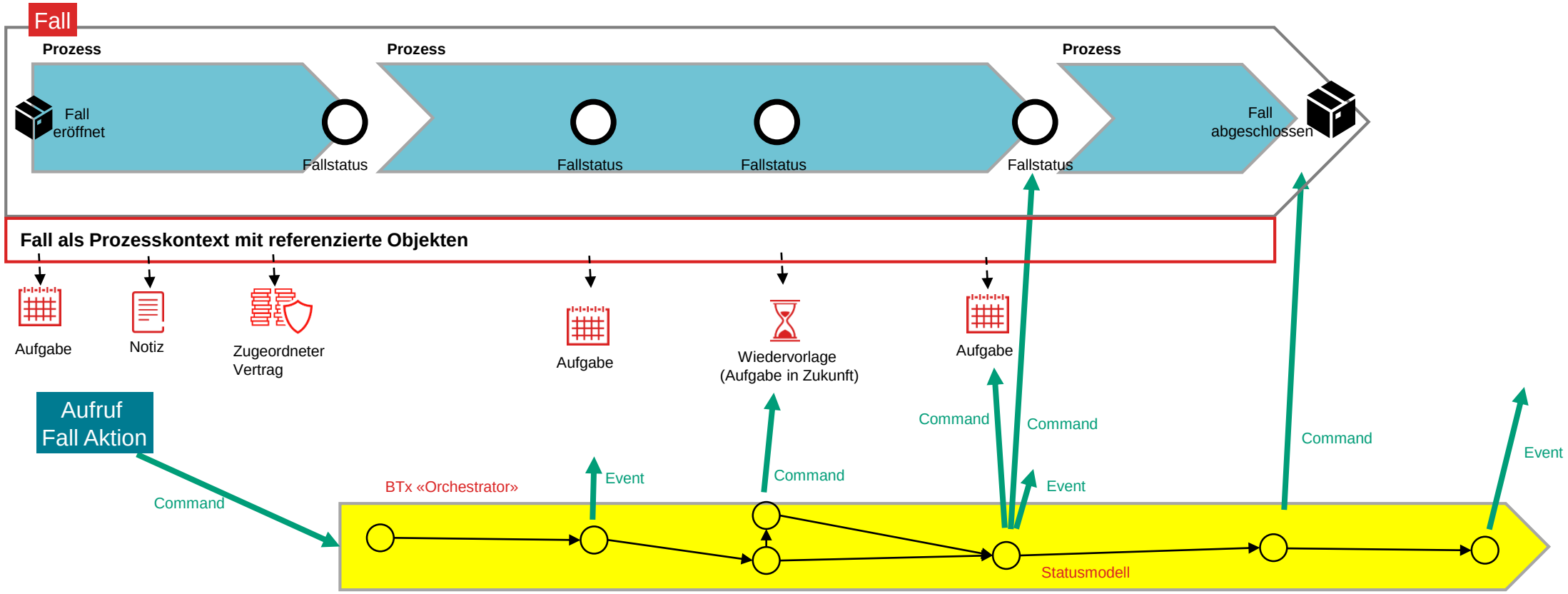
BASE Transaktion Service

Event

Event

Und wo bleiben eigentlich die Benutzer?

Strategie: Fall basiertes Arbeiten

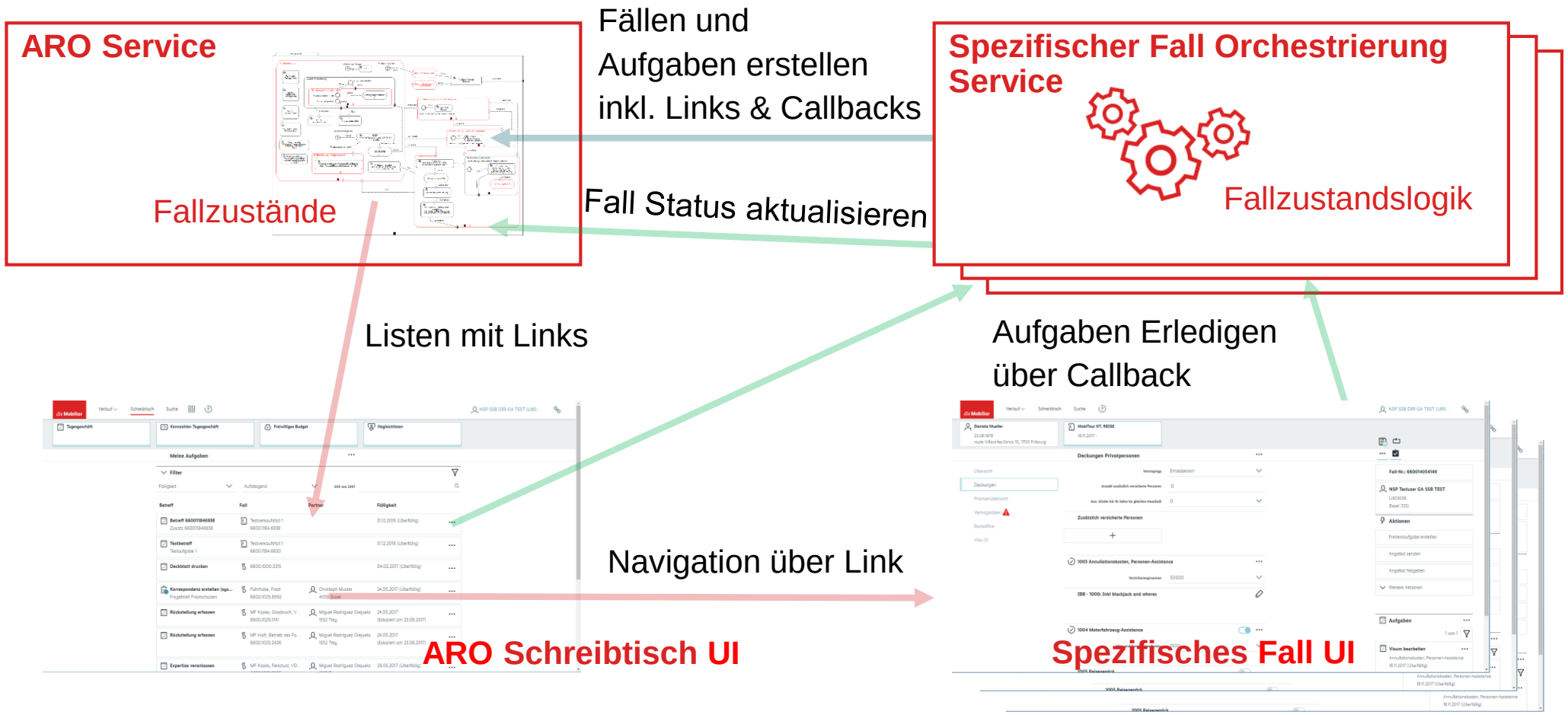


Fälle zum Bearbeiten von Kundenanliegen und internen Prozessen über alle Zugänge hinweg

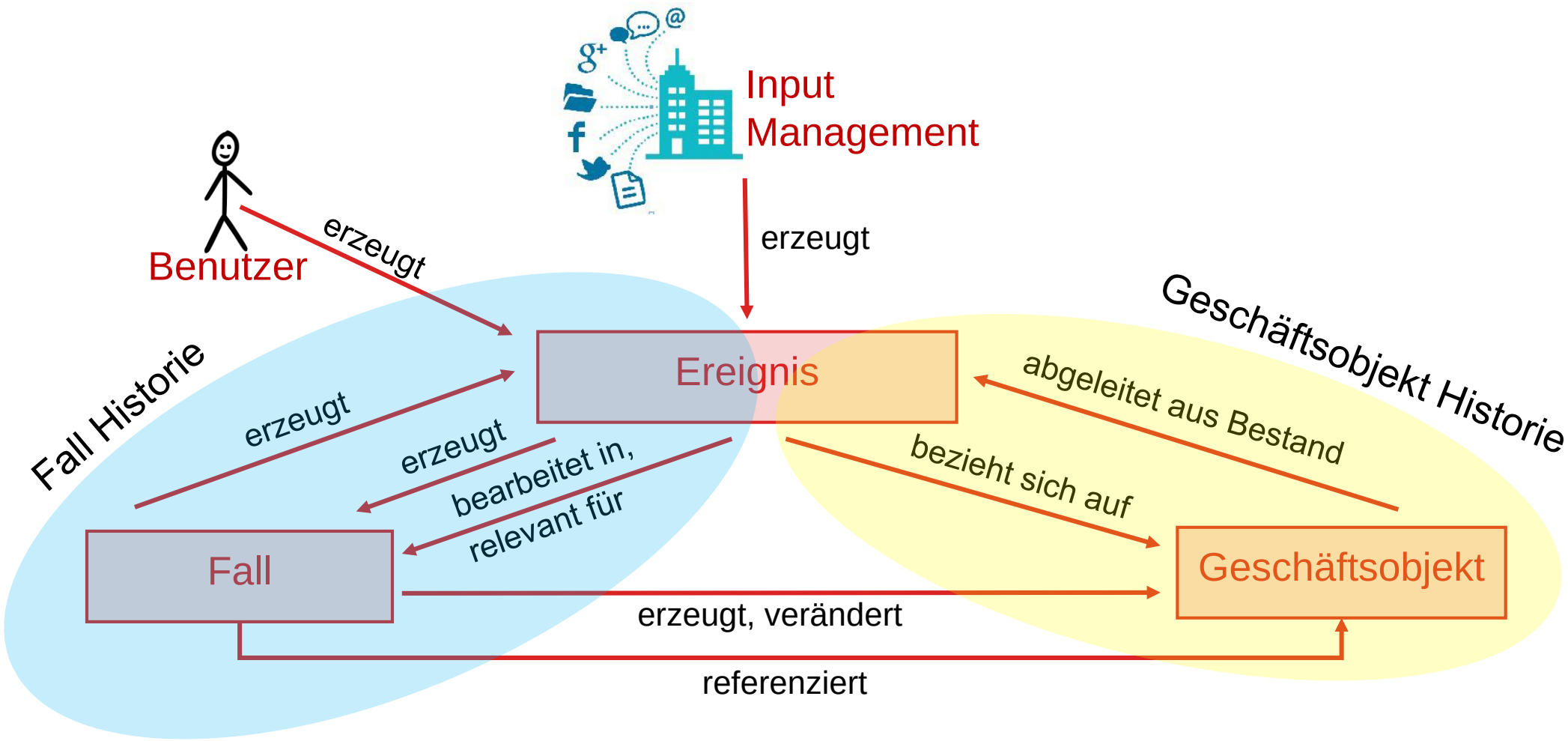
Organisation der Arbeit über Zuweisung von Fällen und Aufgaben inkl. freidefinierbare Arbeitskörbe (persönlich & Team)

Standard Funktionalität für Freitextaufgaben, Aktionen, Notizen und eDossier für Dokumente.

Dezentrale Fall Geschäftslogik & Bearbeitung, zentrale, konsolidierte Übersicht für den Benutzer



Ereignisse als Historie für Fälle und Geschäftsobjekte



Ereignis Historien auf Geschäftsobjekt Übersichten und Fall UIs

B2E Portal | B2E Schreibtisch | B2E Suche | B2E Partner

https://b2e-portal-v2-b.umobi.mobicorp.test/pdv-isp-b2e/pdv/partner/33082107/eventhistory

New Tab | RedNet | die Mobiliar | SAP Portal | Mobiliar CWIKI | JIRA@Mobi | Mobitor | Envchecker | SharePoint | Spezifikationen | SharePoint Houston | Rocket.Chat | BMC | XDMWP | Lossratio | Split.io | B2E Portal | Diverses

die Mobiliar | Schreibtisch | Verlauf | Suche | GA Versicherungs SB (125) T...

Bill Holzhauser
23.04.1990
Lüssliweg 5a, 3604 Thun

- Übersicht
- Partner bearbeiten
- Partnerdetails (Vertriebsplattform)
- Kontaktdaten
- Partnerbeziehungen
- Verträge und Angebote
- Schäden
- Vertragsverlauf öffnen
- Notizen

Historie

Suchen

Anliegen Fall abgeschlossen
TEST CTI B2E GA Schaden, 17.06.2019, 13:56 Uhr

Teilfall wiedereröffnet
TEST CTI B2E GA Schaden, 14.06.2019, 11:11 Uhr

Schadenfall wiedereröffnet
TEST CTI B2E GA Schaden, 14.06.2019, 11:11 Uhr

Eingehender Anruf von +41798357443
Holzhauser Bill
00:06:14
TEST CTI B2E GA Schaden, 13.06.2019, 16:25 Uhr

Eingehender Anruf von +41798357443
Holzhauser Bill
00:01:48
TEST GA Versicherungs SB (125), 13.06.2019, 05:01 Uhr

2 punkt
104499A (Vermittler mit Faktor)
Basel (125)

Historie

Anliegen Fall abgeschlossen
TEST CTI B2E GA Schaden, 17.06.2019, 13:56...

Historie anzeigen (36)

Aktionen
- Korrespondenz erstellen
- Rückrufaufgabe erstellen
- Freitextaufgabe erstellen
- Kontaktnotiz erstellen

Q & A

beat.winistoerfer@mobi.ch